

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Class Test)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT2-CLASS TEST
Question	Q4 — Set Operations on Department and Course Enrollment (1 Mark)

Q4. [1 Mark]

Write a Python program using sets to find:

- (a) Union and intersection of two department lists.
- (b) Students enrolled in Course A but not Course B (set difference).
- (c) Symmetric difference.

Approach / Design

- **Data:** two department lists (dept_cse, dept_ai) and two course enrollment lists (course_a, course_b) are represented as Python sets, since set operations directly model union, intersection, difference, and symmetric difference.
- **Union & intersection:** the | operator combines all unique members of both department sets, while the & operator keeps only members common to both.
- **Set difference:** course_a - course_b returns students in Course A who are not also in Course B.
- **Symmetric difference:** the ^ operator returns students who are in exactly one of the two courses, excluding anyone enrolled in both.
- **Display:** each resulting set is sorted before printing so the output order is consistent and readable.

Python Program

```
"""
CO3-AT2 - Class Test
Q4: Set Operations on Department and Course Enrollment [1 Mark]
-----
Uses sets to find:
(a) Union and intersection of two department lists.
(b) Students enrolled in Course A but not Course B (set difference).
(c) Symmetric difference between Course A and Course B.
"""

# Sample data: two department lists
dept_cse = {"Ravi", "Divya", "Karthik", "Meena"}
dept_ai = {"Meena", "Arun", "Divya", "Priya"}

# Sample data: students enrolled in Course A and Course B
course_a = {"Ravi", "Divya", "Karthik", "Meena"}
course_b = {"Divya", "Meena", "Arun", "Priya"}

# (a) Union and intersection of the two department lists
dept_union = dept_cse | dept_ai
dept_intersection = dept_cse & dept_ai
```

```
# (b) Students enrolled in Course A but not Course B (set difference)
only_course_a = course_a - course_b

# (c) Symmetric difference between Course A and Course B
symmetric_diff = course_a ^ course_b

print("Department Union:", sorted(dept_union))
print("Department Intersection:", sorted(dept_intersection))
print("Enrolled in Course A but not Course B:", sorted(only_course_a))
print("Symmetric Difference (A vs B):", sorted(symmetric_diff))
```

Sample Output

```
Department Union: ['Arun', 'Divya', 'Karthik', 'Meena', 'Priya', 'Ravi']
Department Intersection: ['Divya', 'Meena']
Enrolled in Course A but not Course B: ['Karthik', 'Ravi']
Symmetric Difference (A vs B): ['Arun', 'Karthik', 'Priya', 'Ravi']
```

Conclusion

Representing the department and course rosters as Python sets makes each requirement a single built-in operator: | for union, & for intersection, - for the one-sided difference, and ^ for the symmetric difference. Running the program on the sample rosters correctly merges the CSE and AI department lists, identifies Divya and Meena as common to both departments, finds that Karthik and Ravi are enrolled only in Course A, and shows Arun, Karthik, Priya, and Ravi as enrolled in exactly one of the two courses.